VaLoRe

ML-powered real estate valuation platform

Milestone 1

INFO9023 - Machine Learning Design System

Antoine DECKERS (s2170999)
Hoang Linh BUI (s2503303)
Duy Vu DINH (s2401627)

March 04, 2026

Table of content

1. What is VaLoRe?
2. Milestone 1 Deliverables & System Overview
3. Cloud Configuration
4. Dataset Description
5. EDA
6. Data Preparation
7. Modeling

1. What is *VaLoRe*?

- ML-powered real estate valuation platform.
- **Goal :** build an end-to-end machine learning system that aims to value a residential property given some training data
- **Focus** : system design and engineering components (data pipelines, cloud infrastructure, CI/CD, reproducibility, deployment) rather than model complexity.

2. Milestone 1 - Deliverables (Sprint 1 & 2)

✅ **Set up GitFlow repository structure** (`main, develop, feature/*`)

✅ **Set up CI/CD**: Ruff + pre-commit + pytest + GitHub Actions on PRs

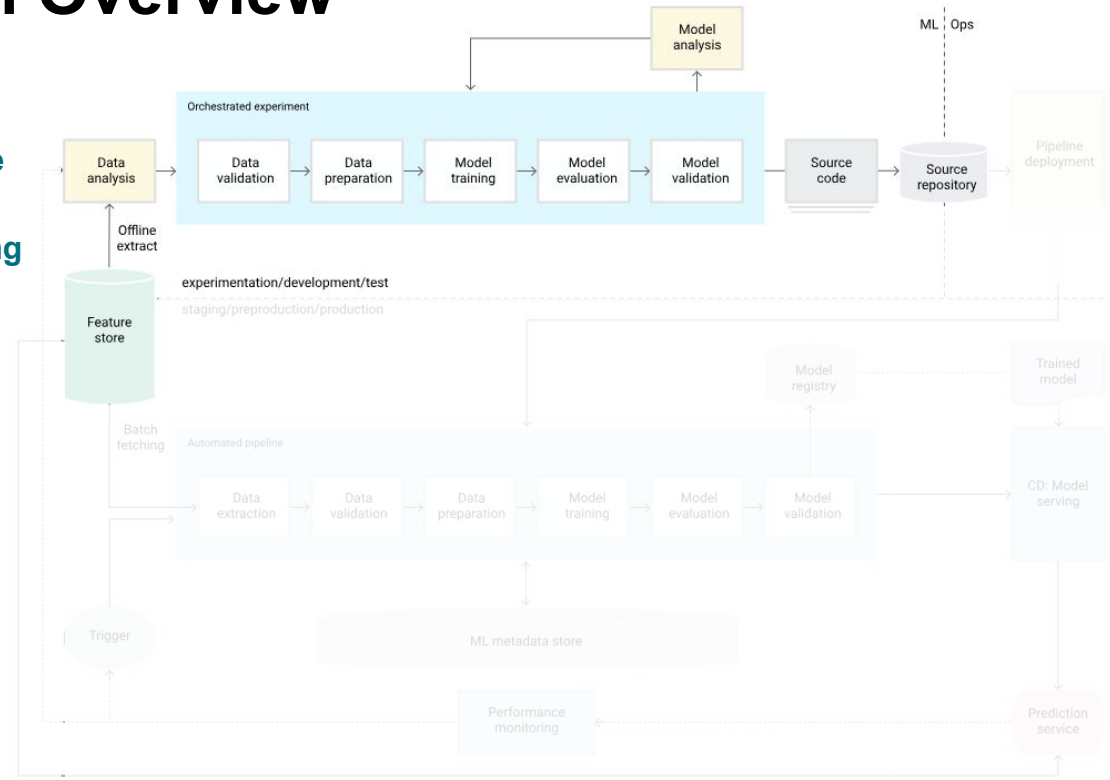
✅ **Store data online** for EDA + training (BigQuery requirement)

✅ **Produce EDA + baseline model**

✅ **Appropriate documentation** (README, docs/feature.md,...)

2. Milestone 1 - System Overview

- Data source/Feature store → **BigQuery**
- Offline extract → **pd Dataframe**
- Data analysis → **EDA**
- Data preparation → **Preprocessing**
- Model training/evaluation → **Baseline**
- Source repository
  - ML → **GitHub repo**
  - OPs → **CI/CD checks**



3. Cloud Configuration

- Google Cloud project (GCP)
  - Project name: `VaLoRe-MLSD-project`
  - Project ID: `valore-mlsd-project`
  - Region: `europe-west1`
  - IAM access for team: **Owner**
- Google Cloud Storage (GCS)
  - Bucket name: `mlsd-valore-2026-0001`
  - Object path (raw): `data/raw/Housing.csv`
- BigQuery:
  - Dataset: `valore`
  - Table: `housing_raw`
- Ingestion
  - Query: CSV (**data**) → **BigQuery**
  - Storage: **files** → **GCS**

4. Dataset Description

- **General overview**
  - Residential property sales in King County, USA between May 2014 and May 2015.
  - **Data type**: tabular / structured
  - **Row count:** 21,614
  - **Number of features:** 21

- **Limitations**
  - **Region:** Limited to King County, Washington State.
  - **Time:** Data represents a snapshot of the 2014-2015 market → price scale factor to adjust to new time period
  - **Provenance:** Random real estate website data mixed.

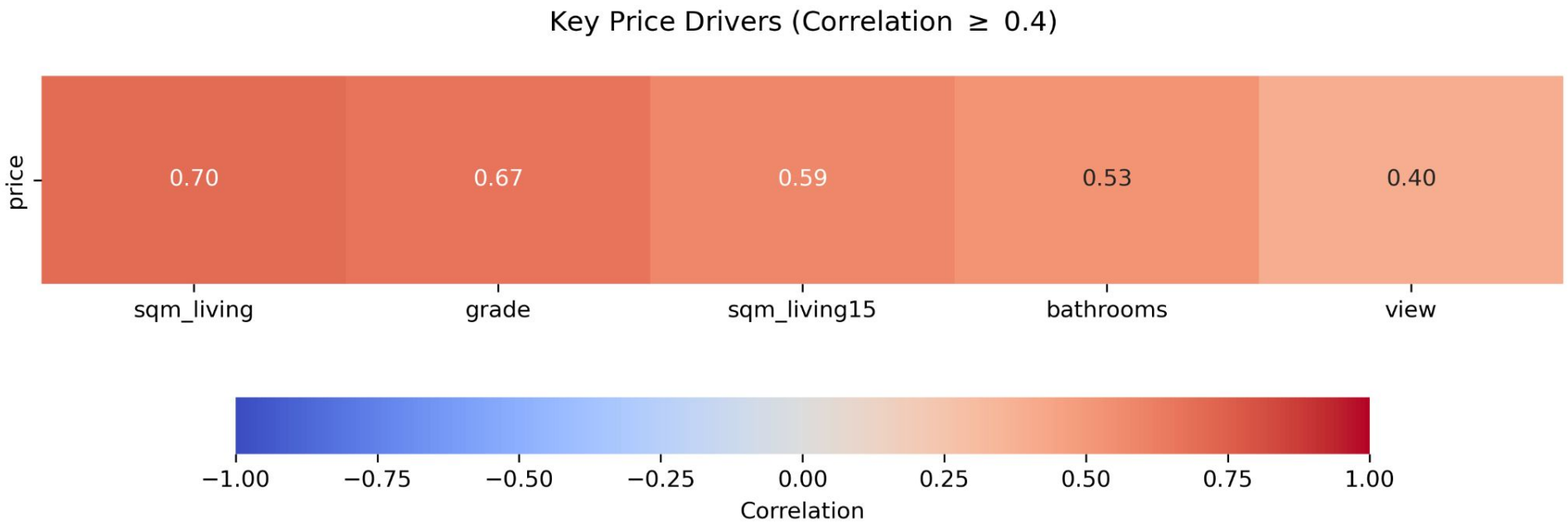
Features summary

Category	Key Features	Statistical Type	Description
Physical Space	`sqft_living`, `sqft_lot`, `sqft_above`, `sqft_basement`	Continuous	Interior and exterior square footage measurements.
Room Counts	`bedrooms`, `bathrooms`, `floors`	Continuous / Discrete	Numerical counts of rooms and property levels.
Quality/View	`grade`, `condition`, `view`, `waterfront`	Ordinal/Binary	Subjective construction, maintenance, and view ratings.
History	`yr_built`, `yr_renovated`	Discrete (Year)	Years of original construction and major updates.
Location	`lat`, `long`, `zipcode`	Geo/Categorical	GPS coordinates and administrative ZIP code.
Context	`sqft_living15`, `sqft_lot15`	Continuous	Average size of the 15 nearest neighbor properties.
Metadata	`price`, `id`, `date`	Target/ID	Sale price (Target), unique ID, and transaction date.

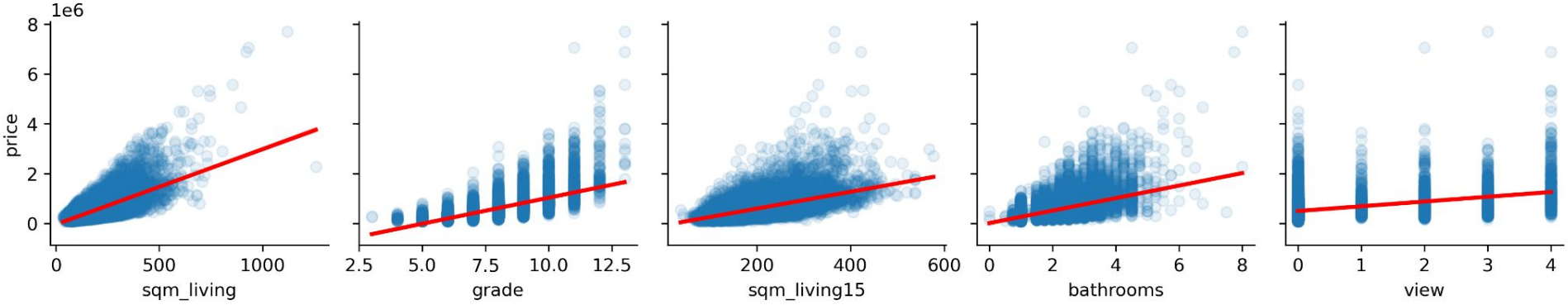
5. EDA

- No missing values / duplicate
- Data time format : The raw date column was provided in an ISO-8601-like string format (20141111T000000). We converted this to a standard datetime format (2014-11-11 00:00:00) to facilitate feature engineering (can extract Year, Month, Day, easily)
- Sqft → sqm for interpretability as europeans

5. EDA


Key Price Drivers (Correlation ≥ 0.4)
price
0.70
0.67
0.59
0.53
0.40
sqm_living
grade
sqm_living15
bathrooms
view
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
Correlation

5. EDA


1e6
price
8
6
4
2
0
0
500
1000
sqm_living
2.5
5.0
7.5
10.0
12.5
grade
200
400
600
sqm_living15
0
2
4
6
8
bathrooms
0
1
2
3
4
view

6. Data preparation before training

- **Variable removal**:
  - Drop id: zero predictive power
  - Drop zipcode: lat / long is more accurate

- **Compositional variables** (add up to a constant)
  - Sqm_living = sqm_above + sqm_basement
  - **Problem :** redundant signal (noise) and can impact model stability / interpretability
  - **Choice :** keep sqm_living and introduce basement_ratio (sqm_basement / sqm_living), drop sqm_basement/above.

7. Modeling - Protocol

- **Data splitting:**

70%	10%	20%
TRAIN	VAL	TEST

1. **Baseline model**: train on **TRAIN** set and evaluate on **VAL** set
2. **Model selection**: **k-fold** cross validation (k=10) on **(TRAIN+VAL)** set → **hyper-param tuning**
3. **Model evaluation**: retrain on **(TRAIN+VAL)** set and evaluate on **TEST** set → **model performance**
4. **Final model**: retrain on **the whole dataset** = (TRAIN+VAL+TEST) set → **deploy/upload to cloud**

- **Evaluation metrics: MAE, RMSE, R2**
- **Regression models**: XGBoost, Random Forest, Linear Regression

Baseline Comparison

Model	MAE	RMSE	R^2
XGBoost	64,279.80	119,566.27	0.8848
Random Forest	70,547.48	135,946.08	0.8511
Linear Regression	125,638.94	188,894.14	0.7126

Hyperparameter Tuning

XGBoost Tuning

Method:

- GridSearchCV
- 10-fold CV
- Scoring: RMSE

Grid:

- n_estimators: [300, 500, 800]
- learning_rate: [0.05, 0.1, 0.2]
- max_depth: [4, 6, 8]

best parameters →

Parameter	Value
learning_rate	0.1
max_depth	4
n_estimators	800

Final Model Performance

Metric	Value
MAE	62,549.77
RMSE	107,557.43
R^2	0.9063

Next steps

- **Migrate dependency management** from pip + requirements.txt to **uv**
- Build a repeatable data **preprocessing pipeline**
  - (cleaning, missing values, encoding)
- **Containerization** + **deployment** (API)
- **Monitoring/logging**
- **Model registry/artifacts** in **GCS**

Thank you for your listening!